1. Токени, словник і невідомі слова

Усний блок: приблизно 40 секунд. Код можна пояснювати за виділеними кроками.

Лабораторна №2 | Конспект для захисту з кодом

Як сказати: Токенізатор ділить текст на фрагменти й переводить їх у числові id. Я порівнюю слова, символьні n-грами, BPE та WordPiece. У послівному підході невідоме слово стає <unk>, тому різні невідомі слова можуть отримати однаковий id.

Поточний код: tokenizer.py, рядок 31

```
def word_tokenize(text):
    """ділить текст на слова, числа і знаки"""
    import re
    return re.findall(r"[^\W\d_]+(?:['’][^\W\d_]+)*|\d+|[^\w\s]|_", text.lower())
```

У регулярному виразі окремо виділяються слова, числа та знаки. Апостроф може залишатися всередині слова. **lower()** зменшує кількість варіантів у словнику, але втрачає початковий регістр.

Поточний код: tokenizer.py, рядок 68

```
def build_word_vocab(texts) -> dict[str, int]:
    """збирає словник зі слів навчального корпусу"""
    words = set()
    for text in texts:
        words.update(word_tokenize(text))
    vocab = {"<pad>": 0, "<unk>": 1}
    for word in sorted(words):
        if word not in vocab:
            vocab[word] = len(vocab)
    return vocab
```

Поточний код: tokenizer.py, рядок 80

```
def encode_words(text, vocab):
    """замінює токени на їхні id"""
    return [vocab.get(word, vocab["<unk>"]) for word in word_tokenize(text)]
```

Що показати: set прибирає повторення; sorted робить id відтворюваними; get повертає id слова або id <unk>. <pad> має id 0 і потрібний для доповнення послідовностей, <unk> має id 1.

Словник містить відповідності токен → id. Embedding-таблиця вже зберігає вектори для цих id: приблизно V·d параметрів, де V - кількість токенів, d - довжина вектора.

Фрагменти взято з поточної гілки lab-2-tokenizers. Повний код і виконані результати є у tokenizer.py та tokenizer.ipynb.

2. N-грами і коротке слово «кіт»

Усний блок: приблизно 60 секунд. Код можна пояснювати за виділеними кроками.

Як сказати: Символьна n-грама - це n сусідніх символів. Фрагменти перекриваються, тому споріднені слова мають спільні n-грами. Короткі фрагменти частіше збігаються, але гірше розрізняють слова; довгі точніші, але частіше невідомі.

Поточний код: tokenizer.py, рядок 119

```
def char_ngrams(word, n=3):
    """ділить слово на фрагменти по n символів"""
    if n <= 0:
        raise ValueError("n має бути більше нуля")
    if not word:
        return []
    marked = "^" + word + "$"
    return [marked[i:i + n] for i in range(len(marked) - n + 1)]
```

Як читається код: додаємо межі ^ і $, потім рухаємо вікно довжини n. Для позначеного рядка довжини L кількість фрагментів - max(0, L - n + 1). Для «кіт» і n=3 виходить [«^кі», «кіт», «іт$»].

Якщо n=6: «кіт» має 3 символи, «^кіт$» - 5. range(5 - 6 + 1) порожній, тому поточна функція повертає []. Це не виняток Python, але слово не отримає представлення.

Можливе покращення, якого зараз немає в лабораторній: fallback для коротких слів

```
if not word:
    return []
marked = "^" + word + "$"
if len(marked) < n:
    return [marked]
return [marked[i:i + n] for i in range(len(marked) - n + 1)]
```

Такий fallback повертає коротший токен, а не справжню шестиграму. Якщо потрібні строго шестиграми, додають спеціальні символи padding. Правило має бути однаковим під час навчання та застосування; словник треба перебудувати.

Поточний код: tokenizer.py, рядок 129

```
def ngram_overlap(word_a, word_b, n=3):
    """порівнює множини n-грам"""
    a = set(char_ngrams(word_a, n))
    b = set(char_ngrams(word_b, n))
    if not a and not b:
        return 1.0
    return len(a & b) / len(a | b)
```

Тут використано **Жаккар**: |A ∩ B| / |A □ B|. Множини втрачають порядок і повторення. Нюанс: дві порожні множини в коді дають 1.0, тому короткі слова без fallback можуть помилково виглядати однаковими.

3. BPE: частоти, злиття і новий текст

Усний блок: приблизно 50 секунд. Код можна пояснювати за виділеними кроками.

Як сказати: BPE починає з окремих символів і зливає найчастішу сусідню пару. Частоти враховують, скільки разів слово було в корпусі. Під час застосування повторюються вивчені злиття. Це навчання за частотами, без градієнтного спуску.

Поточний код: tokenizer.py, рядок 224

```
def most_frequent_pair(vocabulary):
    """шукає найчастішу пару, враховуючи повторення слів"""
    from collections import Counter
    pairs = Counter()
    for symbols, count in vocabulary.items():
        for pair in zip(symbols, symbols[1:]):
            pairs[pair] += count
    if not pairs:
        return None
    return min(pairs, key=lambda pair: (-pairs[pair], pair))
```

Ключові місця: zip утворює сусідні пари; += count додає частоту слова для кожного входження пари. min з ключем (-частота, пара) вибирає найбільшу частоту, а при рівності - лексикографічно найменшу пару.

Поточний код: tokenizer.py, рядок 236

```
def merge_pair(symbols, pair):
    """зливає сусідні елементи вибраної пари"""
    result = []
    i = 0
    while i < len(symbols):
        if i + 1 < len(symbols) and (symbols[i], symbols[i + 1]) == pair:
            result.append(symbols[i] + symbols[i + 1])
            i += 2
        else:
            result.append(symbols[i])
            i += 1
    return tuple(result)
```

Після злиття і збільшується на 2, тому входження не перекриваються. Наприклад, злиття («а», «а») у («а», «а», «а») дасть («аа», «а»), а не два «аа».

Поточний код: tokenizer.py, рядок 311

```
def apply_bpe(word, merges):
    """застосовує злиття в тому порядку, як їх вивчили"""
    if not word:
        return []
    symbols = tuple(word) + ("</w>",)
    for pair in merges:
        symbols = merge_pair(symbols, pair)
    return list(symbols)
```

Маркер </w> позначає кінець слова. BPE з лабораторної працює із символами; якщо застосувати його до нового символу, код може просто повернути цей символ поза навчальним алфавітом. Byte-level BPE натомість має початковий набір усіх 256 байтів.

4. WordPiece: score і найдовший фрагмент

Усний блок: приблизно 60 секунд. Код можна пояснювати за виділеними кроками.

Як сказати: WordPiece у нашій навчальній схемі вибирає пару не лише за частотою. Її score дорівнює freq(a,b)/(freq(a)·freq(b)). Префікс ## позначає продовження слова. Під час застосування шукається найдовший фрагмент зі словника, а не відтворюється список BPE-злиттів.

Поточний код: tokenizer.py, рядок 406

```
def wordpiece_symbols(word):
    """додає ## до символів, які продовжують слово"""
    if not word:
        return ()
    return (word[0],) + tuple("##" + char for char in word[1:])
```

Поточний код: tokenizer.py, рядок 413

```
def wordpiece_pair_scores(vocabulary):
    """рахує оцінку пари за частотами її частин"""
    from collections import Counter
    symbols_count = Counter()
    pairs_count = Counter()
    for symbols, count in vocabulary.items():
        for symbol in symbols:
            symbols_count[symbol] += count
        for pair in zip(symbols, symbols[1:]):
            pairs_count[pair] += count
    return {pair: count / (symbols_count[pair[0]] * symbols_count[pair[1]])
            for pair, count in pairs_count.items()}
```

Частота пари ділиться на частоти обох частин. Рідкісні частини, які зустрічаються разом, можуть мати високий score. Під час злиття внутрішній ## прибирається, а позначка продовження на початку токена зберігається.

Поточний код: tokenizer.py, рядок 467

```
def apply_wordpiece(word, vocab, unk_token="[UNK]"):
    """на кожній позиції бере найдовший відомий фрагмент"""
    pieces = []
    start = 0
    while start < len(word):
        found = None
        for end in range(len(word), start, -1):
            piece = word[start:end]
            if start > 0:
                piece = "##" + piece
            if piece in vocab:
                found = piece
                start = end
                break
        if found is None:
            return [unk_token]
        pieces.append(found)
    return pieces
```

Пояснення циклів: start - початок необробленої частини; end рухається від кінця слова назад. Перший знайдений фрагмент є найдовшим. Якщо знайти фрагмент не вдалося, повертається [UNK] для **всього слова**, а вже знайдені частини відкидаються.

Наприклад, зі словником {«токен», «##ізація», «[UNK]»} слово «токенізація» розкладається, а «токенх» стає [UNK]. Зовнішня нормалізація регістру виконується word_tokenize.

5. Чому BPE стискає краще за наш WordPiece

Усний блок: приблизно 60 секунд. Код можна пояснювати за виділеними кроками.

Алгоритм	V	English	Українська	Грузинська
BPE	500	3.387	4.323	4.928
BPE	1000	2.886	3.668	4.208
BPE	2000	2.469	3.185	3.565
WordPiece	500	4.827	5.945	7.307
WordPiece	1000	4.784	5.810	7.169
WordPiece	2000	4.774	5.412	7.160

Як сказати: BPE пріоритетно зливає масові повторення, тому нові токени помітно скорочують весь корпус. WordPiece-score може вибирати рідкісні фрагменти, які мало змінюють загальну кількість позицій. Тому більший словник у нашому WordPiece майже не знижує fertility. Це результат цього корпусу й навчальної реалізації, а не загальне правило для WordPiece.

Приклад score: пара з частотою 1 і частотами частин 1 та 1 має score 1. Пара з частотою 100 і частотами частин 1000 та 1000 має score 0.0001. Перша може бути обрана раніше, хоча скоротить значно менше входжень.

Поточний код: tokenizer.py, рядок 291

```
def train_bpe(texts, num_merges):
    """навчає послідовність злиттів BPE"""
    if num_merges < 0:
        raise ValueError("кількість злиттів не може бути від’ємною")
    vocabulary = {}
    for word, count in word_frequencies(texts).items():
        vocabulary[tuple(word) + ("</w>",)] = count
    state = _training_counts(vocabulary)
    pair_counts = state[2]
    merges = []
    for _ in range(num_merges):
        if not pair_counts:
            break
        pair = min(pair_counts, key=lambda p: (-pair_counts[p], p))
        _merge_training_pair(state, pair, merge_pair)
        merges.append(pair)
    return merges
```

Що означають допоміжні функції: _training_counts готує частоти та індекси слів із кожною парою; state[2] - частоти пар. _merge_training_pair оновлює лише змінені слова. Це прискорює підрахунок, але не змінює правило вибору злиття.

WordPiece-словник збільшився до заданих 500/1000/2000 токенів. Стабільна частка [UNK] не доводить помилку навчання: за умовою числа й пунктуація не входять у навчання словника, тому самі нові злиття букв їх не додають.

6. Fertility, однакові речення й чесність порівняння

Усний блок: приблизно 40 секунд. Код можна пояснювати за виділеними кроками.

Як сказати: Fertility - це кількість токенів, поділена на кількість слів. Я рахую відношення сум по всьому корпусу, а не середнє по реченнях. Нижча fertility означає коротшу послідовність, але не гарантує кращого збереження інформації.

Поточний код: tokenizer.py, рядок 722

```
def fertility(tokenize, texts):
    """ділить загальну кількість токенів на кількість слів"""
    total_tokens = 0
    total_words = 0
    for text in texts:
        total_tokens += len(tokenize(text))
        total_words += len(text.split())
    if total_words == 0:
        raise ValueError("у корпусі немає слів")
    return total_tokens / total_words
```

Слова тут - непорожні фрагменти після str.split(). tokenize(text) повертає токени конкретного токенізатора. Якщо слів немає, ділення не виконується: видається ValueError.

Чому суми: 10 слів → 20 токенів, а 90 слів → 90 токенів. Правильне значення (20+90)/(10+90)=1.1. Середнє двох значень (2+1)/2=1.5 дало б короткому реченню завелику вагу.

Дані FLORES dev	Речень
Англійська	997
Українська	997
Грузинська	997
Разом	2991

Навіщо порівну: мови мають однакову кількість навчальних прикладів. Речення паралельні, тобто передають однаковий зміст, тому менше впливає тематика. Але кількість слів і символів усе одно різна - це не повна рівність умов.

Обмеження: власні словники навчалися й оцінювалися на тому самому dev-корпусі. Це train fertility. Наш код знижує регістр, а GPT зберігає його; GPT-словники також набагато більші. Для узагальнення потрібні окремі тестові тексти.

Чому [UNK] важливий: WordPiece замінив 13.22% / 18.24% / 19.01% вхідних одиниць word_tokenize для English / української / грузинської. Тут враховано і числа та пунктуацію. Один [UNK] займає мало місця, але втрачає слово. Послівний словник має 21352 елементи, триграмний - 15649; нуль OOV на train очікуваний.

7. GPT і головне питання про українську

Усний блок: приблизно 60 секунд. Код можна пояснювати за виділеними кроками.

Токенізатор	English	Українська	Грузинська
GPT-2	1.228	8.024	21.545
GPT-3.5 / GPT-4	1.231	4.202	15.345
GPT-5	1.219	2.442	2.780
BPE-2000	2.469	3.185	3.565
WordPiece-2000	4.774	5.412	7.160

Як сказати: GPT-2 має набагато вищу fertility для української через покриття словника, особливості мови та байтове представлення. Сказати лише «через відмінки» недостатньо. У нашій таблиці GPT-5 дає найкоротші послідовності, але ми перевіряли токенізатори, а не якість відповідей моделей.

1. Покриття словника. GPT-2 краще пристосований до англомовних фрагментів. Часті англійські слова й частини слів частіше вже є готовими токенами; українські частіше доводиться дробити.

2. Словозміна. В українській більше форм через відмінки, числа, роди та узгодження: студент, студента, студентові, студентами. Частоти розподіляються між цими формами. В англійській словозміни загалом менше; неправильно казати, що там узагалі немає відмінків або форм слова.

3. Байти UTF-8. Англійська ASCII-літера займає 1 байт, українська літера зазвичай 2, грузинська - 3. GPT-2 використовує byte-level BPE. Без потрібних злиттів текст ділиться на дрібні байтові фрагменти. **Два байти не означають автоматично два токени:** BPE може об'єднати багато байтів в один токен.

Відповідності, які фактично використав tiktoken у цій лабі

```
GPT-2          → gpt2
GPT-3.5 / GPT-4 → cl100k_base
GPT-5          → o200k_base
```

GPT-3.5 та GPT-4 мають однакові результати, бо використовують один encoding. GPT-5 у досліді краще покриває українську й грузинську. Більша fertility означає довший токенний вхід і менше тексту в контексті того самого розміру, але сама по собі не вимірює розуміння мови.

Питання про нового символа: byte-level BPE може зібрати його з байтів без [UNK]. Наш символьний BPE повертає символ, навіть якщо його не було в навчальному алфавіті; це ще не гарантія наявності id у фіксованому словнику.

Джерела: WordPiece: Hugging Face; GPT-2 encoder; GPT-2 model card; шаблон лабораторної. Код і числа - з локальних tokenizer.py та виконаного tokenizer.ipynb.